Voice Assistance for Multimedia Control

Voice-Controlled Media Playback System (Python; tested on a laptop, Raspberry Pi-ready)

Version 1.1, 27 October 2026 (Version 1.0: 28 September 2026)

Aaditya Vashisht: PES1UG24CS004
Aarohi Jaiswal: PES1UG24CS009
Abhay S: PES1UG24CS013

Revision History

Version	Date	Changes	Author
1.0	28 Sep 2026	Initial version, submitted with the Test Plan and Architecture document.	Team
1.1	27 Oct 2026	Aligned with the system built in Sprints 1 and 2. Each change matches a deviation in the project README (D-x): • Platform: tested on a laptop instead of a Raspberry Pi / ESP32 board (D-1); NFR-5 rewritten; ESP32 out of scope. • Media player: YouTube in a Google Chrome window controlled through Selenium (D-2). • Speech recognition: offline Vosk library; cloud recognition only when explicitly selected (D-3); SR-2 wording updated. • NFR-3: continuous operation reduced from 8 hours to 2 hours, approved by the instructor on 27 Oct 2026 (D-5). FR-1 to FR-10, NFR-1, NFR-2, NFR-4, NFR-6, SO-1 and SR-1 are unchanged. Changed text is marked “Revised in v1.1”.	Abhay S

1. Software Requirements Specification

1. Introduction

1.1 Purpose

This document specifies the software requirements for a Voice Assistance system for Multimedia Control. The system allows a user to control media playback using spoken voice commands, processed by Python software running on the user’s computer (a laptop in this version; a Raspberry Pi is a supported target), instead of physical buttons or a mouse/keyboard. This SRS is intended for the project development team, the course evaluator, and any future maintainer of the system. *[Revised in v1.1: platform, D-1]*

1.2 Scope

The system, referred to hereafter as the “Voice Media Controller”, will:

- Continuously listen for the wake word “Hey Media” through the computer’s microphone.
- Recognize a fixed set of multimedia control commands: Play, Pause, Resume, Next, Previous, Volume Up, and Volume Down.

- Execute the recognized command on a media player: YouTube playing in a Google Chrome window controlled by the system. *[Revised in v1.1: D-2]*
- Run on a standard laptop (Windows, macOS or Linux) using Python and the offline Vosk speech-recognition library; the same software can run on a Raspberry Pi (Linux) without redesign. *[Revised in v1.1: D-1, D-3]*

The system will NOT perform general-purpose conversational AI, internet search, smart-home control, or support commands outside the defined multimedia control set in this version.

1.3 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
ASR	Automatic Speech Recognition
Wake Word	The phrase “Hey Media” that activates command listening
Listening window	The 4-second period after the wake word in which one command is accepted (added in v1.1)
Media Player	The playback application controlled by the system: YouTube in a Google Chrome window (revised in v1.1)
Vosk	Offline speech-recognition library that runs on the device without sending audio to the internet (added in v1.1)
Selenium	Browser-automation library used to control the Chrome window (added in v1.1)
Pi	Raspberry Pi single-board computer (supported target)

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- Raspberry Pi Foundation, Raspberry Pi Documentation, www.raspberrypi.com/documentation
- Alpha Cephei, Vosk Speech Recognition Toolkit, alphacephei.com/vosk (added in v1.1)
- Selenium Project, Selenium WebDriver Documentation, www.selenium.dev/documentation (added in v1.1)
- Project handouts: SE Mini-Project Deliverables Part-1 and Part-2.

1.5 Overview

The remainder of this document is organized as follows: Section 2 gives an overall description of the product, its perspective, functions, users and constraints. Section 3 lists the specific functional and non-functional requirements along with the use case diagram. Section 4 details the security objectives and requirements for the system. Changes from version 1.0 are listed in the Revision History and marked in the text.

2. Overall Description

2.1 Product Perspective

The Voice Media Controller is a software add-on that runs on the same computer as the media player. It listens for audio input through the computer's microphone and controls YouTube in a dedicated Chrome window. It is not a replacement for the media player itself, only a hands-free control layer on top of it. *[Revised in v1.1: D-1, D-2]*

2.2 Product Functions

At a high level, the system:

- Captures live audio from the computer's microphone. *[Revised in v1.1: D-1]*
- Converts speech to text locally using the Vosk ASR library, and accepts a command only after the wake word, within the listening window. *[Revised in v1.1: D-3]*
- Maps the recognized command to one of: Play, Pause, Resume, Next, Previous, Volume Up, Volume Down.
- Sends the corresponding control action to the YouTube player. *[Revised in v1.1: D-2]*
- Provides basic feedback (console messages and audible tones) confirming the command executed or was rejected.

2.3 User Characteristics

The primary user is a single end user operating the media player hands-free, for example while cooking, exercising, or working. No technical expertise is required to use the system. Only the developer/maintainer requires technical knowledge of Python, speech recognition and browser automation.

2.4 Constraints

- Limited to the fixed command vocabulary defined in Section 1.2; free-form conversation is out of scope.
- Runs on a computer with Python 3.9 or newer. Available processing power limits the choice of ASR model, so a small offline model is used rather than a large cloud model. *[Revised in v1.1: D-1, D-3]*
- Requires a working microphone. Speech recognition needs no network connection; playing YouTube videos requires an internet connection. *[Revised in v1.1: D-2]*
- Implementation language restricted to Python. *[Revised in v1.1: ESP32 firmware out of scope]*

2.5 Assumptions and Dependencies

- Google Chrome is installed and can be controlled by Selenium; the Previous command is used while a YouTube playlist is playing. *[Revised in v1.1: D-2]*
- The operating environment has low to moderate background noise.
- The required Python libraries (Vosk, Selenium, sounddevice) are installable, and the Vosk speech model is downloaded once before first use. *[Revised in v1.1: D-3]*
- A single user issues commands at a time; multi-user concurrent commands are not handled in this version.

3. Specific Requirements

3.1 Functional Requirements

Functional requirements are written to be clear, unambiguous, measurable, and independently testable, in line with IEEE 830 guidelines. They are unchanged from version 1.0.

ID	Requirement	Description / Acceptance Criteria
FR-1	Wake / Listen	The system shall continuously monitor the microphone input and begin command processing within 1 second of detecting the wake word.
FR-2	Play Command	The system shall start media playback within 2 seconds of recognizing the spoken command “Play”.
FR-3	Pause Command	The system shall pause the currently playing media within 2 seconds of recognizing the spoken command “Pause”.
FR-4	Resume Command	The system shall resume previously paused media within 2 seconds of recognizing the spoken command “Resume”.
FR-5	Next Track Command	The system shall skip to the next media item within 2 seconds of recognizing the spoken command “Next”.
FR-6	Previous Track Command	The system shall return to the previous media item within 2 seconds of recognizing the spoken command “Previous”.
FR-7	Volume Up Command	The system shall increase playback volume by a fixed step (10%) within 1 second of recognizing the spoken command “Volume Up”.
FR-8	Volume Down Command	The system shall decrease playback volume by a fixed step (10%) within 1 second of recognizing the spoken command “Volume Down”.
FR-9	Command Rejection	The system shall ignore and discard any audio input that does not match a defined command in the vocabulary, without executing an action.
FR-10	Feedback	The system shall provide a visible or audible acknowledgement (e.g. LED blink or log message) each time a command is successfully executed.

3.2 Non-Functional Requirements

ID	Requirement	Description / Acceptance Criteria
NFR-1	Performance	The system shall recognize and execute a valid voice command within 2 seconds under normal (low) background noise, measured from end of utterance to action execution.
NFR-2	Reliability	The system shall correctly recognize valid commands with at least 90% accuracy across 50 test utterances in a quiet room.
NFR-3	Availability	The system shall remain in a listening-ready state for at least 2 continuous hours of operation without requiring a restart. *[Revised in v1.1: reduced from 8 hours, approved by the instructor on 27 Oct 2026, D-5]*

ID	Requirement	Description / Acceptance Criteria
NFR-4	Usability	A first-time user shall be able to issue a working command without reading a manual, using only the 7 defined command words.
NFR-5	Portability	The core recognition and control software shall run on Windows, macOS and Linux computers with Python 3.9 or newer, including a Raspberry Pi (Linux), without code changes. ESP32 support is out of scope for this version. *[Revised in v1.1: D-1]*
NFR-6	Maintainability	Each voice command shall be defined as a separate, independently testable function/module so new commands can be added without modifying existing command logic.

3.3 Use Case Diagram

Figure 1 shows the primary actor (User) and the system's use cases. All direct commands include the "Recognize Voice Command" use case, which in turn includes "Authenticate Command Source" to satisfy the security requirements in Section 4. The diagram is unchanged from version 1.0.

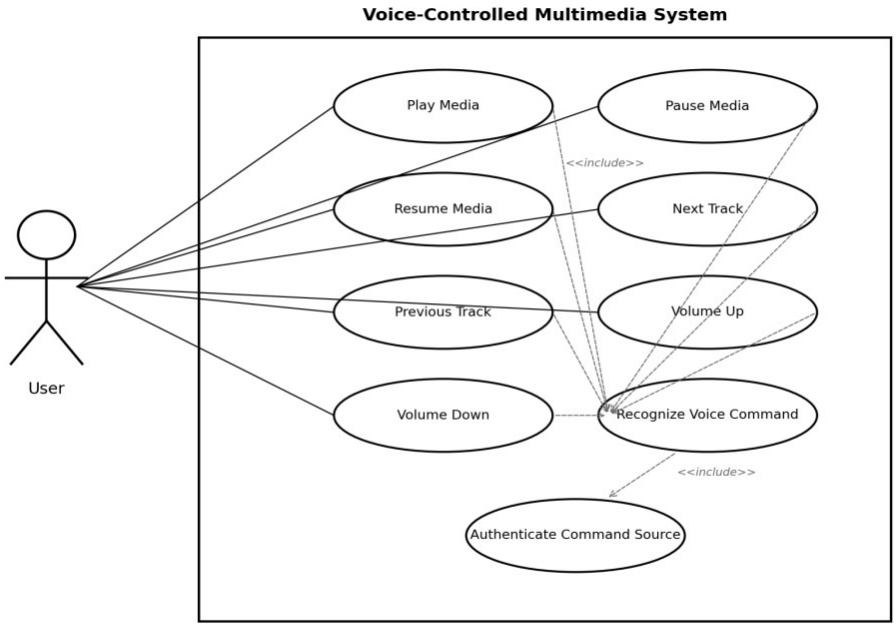


Figure 1: UML Use Case Diagram — Voice-Controlled Multimedia System

4. Security Requirements

4.1 Security Objectives

ID	Objective	Description
SO-1	Command Source Integrity	Ensure that multimedia control commands are only accepted from the intended, authorized microphone/user, preventing accidental or malicious triggering from other audio sources (e.g. a TV or radio in the background).
SO-2	Communication Confidentiality	Protect any command or audio data passed between the microphone, the processing software and the media player on the computer running the system from interception or tampering. *[Revised in v1.1: Pi/ESP replaced by the computer running the system, D-1]*

4.2 Security Requirements

ID	Requirement	Description
SR-1	Wake-Word Gating	The system shall require detection of a specific wake word before accepting any control command, reducing the chance of unintended activation by background audio.
SR-2	Local Processing	The system shall process voice commands locally on the device running the system, without transmitting raw audio to external/cloud services. A cloud recognizer may be used only when explicitly selected by the user. *[Revised in v1.1: D-3]*